Building a PDF Generator

with AWS Lambda & Java (Step-by-Step Guide)

This guide walks through building a small AWS Lambda function in Java that generates a PDF file using Apache PDFBox, packages it as a deployable JAR, and deploys/tests it on AWS — using both the AWS Console and the AWS CLI.

What you'll build:	A Lambda function that accepts a JSON request (name/message), generates a PDF in memory, and
Language:	Java 11/17 (Amazon Corretto runtime)
Libraries:	Apache PDFBox 2.x, AWS Lambda Java Core/Events, AWS SDK v2 (S3, optional)
Build tool:	Apache Maven

Prerequisites

- An **AWS account** with permission to create Lambda functions, IAM roles, and (optionally) S3 buckets.
- **Java JDK 11 or 17** installed locally.
- **Apache Maven** installed (`mvn -v` to verify).
- **AWS CLI v2** installed and configured (`aws configure`) — optional but recommended.
- Basic familiarity with the terminal / command line.

Note: If you prefer, every CLI step below can also be done through the AWS Console UI — the console equivalents are called out where relevant.

STEP 1 Create the Maven Project

Create a new folder and initialize a Maven project:

```
mkdir lambda-pdf-generator
cd lambda-pdf-generator
mvn archetype:generate \
    -DgroupId=com.example.pdf \
    -DartifactId=lambda-pdf-generator \
    -DarchetypeArtifactId=maven-archetype-quickstart \
    -DinteractiveMode=false
cd lambda-pdf-generator
```

Your folder structure should look like this:

```
lambda-pdf-generator/
■■■ pom.xml
■■■ src
    ■■■ main
        ■■■ java
            ■■■ com/example/pdf/App.java
```

STEP 2 Configure pom.xml

Replace the generated `pom.xml` with the version below. It adds the AWS Lambda Java libraries, Apache PDFBox for PDF creation, and the `maven-shade-plugin` needed to build a single "fat JAR" containing all dependencies (required for Lambda deployment).

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example.pdf</groupId>
  <artifactId>lambda-pdf-generator</artifactId>
  <version>1.0</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
```

```
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <!-- AWS Lambda core interfaces (RequestHandler, Context) -->
    <dependency>
      <groupId>com.amazonaws</groupId>
      <artifactId>aws-lambda-java-core</artifactId>
      <version>1.2.3</version>
    </dependency>
    <!-- Lambda event classes (APIGatewayProxyRequestEvent, etc.) -->
    <dependency>
      <groupId>com.amazonaws</groupId>
      <artifactId>aws-lambda-java-events</artifactId>
      <version>3.11.4</version>
    </dependency>
    <!-- PDF creation -->
    <dependency>
      <groupId>org.apache.pdfbox</groupId>
      <artifactId>pdfbox</artifactId>
      <version>2.0.30</version>
    </dependency>
    <!-- JSON parsing -->
    <dependency>
      <groupId>com.google.code.gson</groupId>
      <artifactId>gson</artifactId>
      <version>2.10.1</version>
    </dependency>
    <!-- (Optional) AWS SDK v2 S3 client, if uploading generated PDFs to S3 -->
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>s3</artifactId>
      <version>2.25.16</version>
    </dependency>
  </dependencies>

  <build>
    <finalName>lambda-pdf-generator</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-shade-plugin</artifactId>
        <version>3.5.1</version>
        <executions>
          <execution>
            <phase>package</phase>
            <goals><goal>shade</goal></goals>
            <configuration>
              <createDependencyReducedPom>false</createDependencyReducedPom>
            </configuration>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
</project>
```

STEP 3 Write the Lambda Handler

Delete the sample `App.java` and create `src/main/java/com/example/pdf/PdfHandler.java` with the following content. This handler:

- Reads a JSON input like `{"name": "Alice", "message": "Hello!"}`
- Builds a one-page PDF in memory using PDFBox
- Returns the PDF as a Base64-encoded string in the response body

```
package com.example.pdf;

import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.APIGatewayProxyRequestEvent;
import com.amazonaws.services.lambda.runtime.events.APIGatewayProxyResponseEvent;
import com.google.gson.Gson;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.PDPage;
import org.apache.pdfbox.pdmodel.PDPageContentStream;
import org.apache.pdfbox.pdmodel.font.PDType1Font;

import java.io.ByteArrayOutputStream;
import java.util.Base64;
import java.util.HashMap;
import java.util.Map;

public class PdfHandler implements
        RequestHandler<APIGatewayProxyRequestEvent, APIGatewayProxyResponseEvent> {

    static class Input {
        String name;
        String message;
    }

    @Override
    public APIGatewayProxyResponseEvent handleRequest(
            APIGatewayProxyRequestEvent request, Context context) {

        context.getLogger().log("Received request body: " + request.getBody());

        Gson gson = new Gson();
        Input input = gson.fromJson(request.getBody(), Input.class);
        String name = (input != null && input.name != null) ? input.name : "there";
        String message = (input != null && input.message != null)
                ? input.message : "This PDF was generated by AWS Lambda!";

        try {
            byte[] pdfBytes = generatePdf(name, message);
            String base64Pdf = Base64.getEncoder().encodeToString(pdfBytes);

            Map<String, String> headers = new HashMap<>();
            headers.put("Content-Type", "application/pdf");
            headers.put("Content-Disposition", "attachment; filename=\"greeting.pdf\"");

            APIGatewayProxyResponseEvent response = new APIGatewayProxyResponseEvent();
            response.setStatusCode(200);
            response.setHeaders(headers);
            response.setIsBase64Encoded(true);
            response.setBody(base64Pdf);
            return response;
```

```
            } catch (Exception e) {
                context.getLogger().log("Error generating PDF: " + e.getMessage());
                APIGatewayProxyResponseEvent errorResponse = new APIGatewayProxyResponseEvent();
                errorResponse.setStatusCode(500);
                errorResponse.setBody("Error generating PDF: " + e.getMessage());
                return errorResponse;
            }
        }

        private byte[] generatePdf(String name, String message) throws Exception {
            try (PDDocument document = new PDDocument()) {
                PDPage page = new PDPage();
                document.addPage(page);

                try (PDPageContentStream content = new PDPageContentStream(document, page)) {
                    content.beginText();
                    content.setFont(PDType1Font.HELVETICA_BOLD, 20);
                    content.newLineAtOffset(60, 700);
                    content.showText("Hello, " + name + "!");
                    content.endText();

                    content.beginText();
                    content.setFont(PDType1Font.HELVETICA, 13);
                    content.newLineAtOffset(60, 660);
                    content.showText(message);
                    content.endText();
                }

                ByteArrayOutputStream out = new ByteArrayOutputStream();
                document.save(out);
                return out.toByteArray();
            }
        }
```

Note: The `PDDocument`, page, and content stream are all closed inside try-with-resources blocks — this matters in Lambda because leaked resources can persist across warm invocations of the same execution environment.

STEP 4 Build the Deployable JAR

From the project root, run:

```
mvn clean package
```

This produces the fat JAR at:

```
target/lambda-pdf-generator.jar
```

Verify the JAR contains your dependencies (PDFBox, Gson, etc.) — it should be several megabytes in size, not a few kilobytes.

STEP 5 Create an IAM Execution Role

Lambda needs a role granting it permission to write logs (and S3 access, if used). Using the AWS CLI:

```
cat > trust-policy.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{
```

```
    "Effect": "Allow",
    "Principal": {"Service": "lambda.amazonaws.com"},
    "Action": "sts:AssumeRole"
  }]
}
EOF

aws iam create-role \
  --role-name lambda-pdf-generator-role \
  --assume-role-policy-document file://trust-policy.json

aws iam attach-role-policy \
  --role-name lambda-pdf-generator-role \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
```

Console alternative: IAM → Roles → Create role → Trusted entity: *AWS service* → *Lambda* → attach policy *AWSLambdaBasicExecutionRole* → name it `lambda-pdf-generator-role`.

STEP 6 Create the Lambda Function

Option A — AWS CLI

```
aws lambda create-function \
  --function-name pdf-generator \
  --runtime java11 \
  --role arn:aws:iam::<YOUR_ACCOUNT_ID>:role/lambda-pdf-generator-role \
  --handler com.example.pdf.PdfHandler::handleRequest \
  --zip-file fileb://target/lambda-pdf-generator.jar \
  --timeout 15 \
  --memory-size 512
```

Option B — AWS Console

- Go to **Lambda** → **Create function** → **Author from scratch**.
- Function name: `pdf-generator`. Runtime: **Java 11** (or Java 17).
- Under *Change default execution role*, choose *Use an existing role* and select `lambda-pdf-generator-role`.
- Click **Create function**.
- In the *Code* tab, click **Upload from** → **.zip or .jar file** and upload `target/lambda-pdf-generator.jar`.
- In *Runtime settings*, set **Handler** to `com.example.pdf.PdfHandler::handleRequest`.
- In *Configuration* → *General configuration*, bump **Memory** to at least 512 MB and **Timeout** to 15 seconds (PDFBox's first cold-start can be slow).

STEP 7 Test the Function

Console test

- Open the function → **Test** tab → create a new test event.
- Use this event JSON (it simulates an API Gateway request):

```
{
  "body": "{\"name\": \"Alice\", \"message\": \"Thanks for trying Lambda + PDFBox!\"}"
}
```

Click **Test**. A successful run returns a 200 status with a long Base64 string in the body — that's your PDF.

CLI test

```
aws lambda invoke \
  --function-name pdf-generator \
  --payload '{"body": "{\"name\": \"Alice\", \"message\": \"Hi from Lambda\"}"}' \
  --cli-binary-format raw-in-base64-out \
  response.json

cat response.json
```

Decode the result to an actual PDF file

```
python3 -c "
import json, base64
data = json.load(open('response.json'))
pdf_bytes = base64.b64decode(data['body'])
open('output.pdf', 'wb').write(pdf_bytes)
"
# output.pdf now opens like any normal PDF
```

STEP 8 (Optional) Expose via API Gateway

To call this Lambda over HTTP instead of the CLI/console:

- Go to **API Gateway** → **Create API** → **HTTP API**.
- Add an integration pointing to your `pdf-generator` Lambda function.
- Create a route, e.g. `POST /generate-pdf`.
- Deploy the API to a stage (e.g. `$default`).
- Call it with curl:

```
curl -X POST https://<api-id>.execute-api.<region>.amazonaws.com/generate-pdf \
  -H "Content-Type: application/json" \
  -d '{"name":"Alice","message":"Hello from the web!"}' \
  --output greeting.pdf
```

STEP 9 (Optional) Save the PDF to S3 Instead of Returning It

For larger PDFs, returning Base64 through API Gateway (6 MB payload limit) isn't ideal. Instead, upload to S3 and return a pre-signed URL. Add this to the handler after generating `pdfBytes`:

```
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.PutObjectRequest;
import software.amazon.awssdk.core.sync.RequestBody;

S3Client s3 = S3Client.builder().build();
String key = "generated/" + java.util.UUID.randomUUID() + ".pdf";
```

```
        s3.putObject(
            PutObjectRequest.builder()
                .bucket("your-bucket-name")
                .key(key)
                .contentType("application/pdf")
                .build(),
            RequestBody.fromBytes(pdfBytes)
        );
        // Then generate a pre-signed GET URL (via S3Presigner) and return that URL
```

Note: If you use S3, remember to grant the Lambda role `s3:PutObject` (and `s3:GetObject` for presigning) on that bucket.

Troubleshooting Checklist

- **"Class not found" errors:** the shaded JAR wasn't built — re-run `mvn clean package` and confirm the JAR size is a few MB, not KB.
- **Handler not found:** double-check the handler string matches `package.ClassName::methodName` exactly.
- **Timeout errors:** raise Lambda's timeout/memory — PDFBox cold starts can take a few seconds.
- **Corrupted PDF when testing via curl/API Gateway:** make sure `isBase64Encoded` is set to `true` and API Gateway's binary media types include `application/pdf`.
- **Permission denied writing to S3:** check the IAM role's attached policies include S3 access.

You now have a working AWS Lambda function, in Java, that generates PDFs on demand — testable from the console, the CLI, or over HTTP via API Gateway.